How to avoid hardcoding **access_key** & **secret_key** in terraform configuration files?

Hardcoding AWS access keys in Terraform files exposes sensitive information to security risks and makes credential management difficult, especially when rotating keys.

Using environment variables or secret management tools ensures secure, scalable, and automated handling of credentials.

Option1: **Use Environment Variables**

Terraform can automatically use AWS credentials from environment variables, which eliminates the need to hardcode them.

Set the following environment variables:

```
export AWS_ACCESS_KEY_ID="<your-access-key-id>"
export AWS_SECRET_ACCESS_KEY="<your-secret-access-key>"
```

if we set above variable ***terraform will automatically detect*** and use these variables during the execution.

Option2: **Use the AWS CLI Configuration File**

Install aws cli & run **`aws configure`** command

provide **access_key** & **secret_key** then ***terraform will automatically detect*** and use these variables during the execution.

Other options:

Using profile or storing credentials in a files (or) by defining them as variables

Recommended Approach:

Is to use Environment Variables, **IAM roles & Profiles** are the best practices for securely managing AWS credentials in Terraform.

We can also Use tools like **AWS Vault**.

Ex2. Write an terraform file to Create a 2 VM in aws cloud (ec2 instance)

```
provider "aws" {
  region  = "us-west-2"
}

resource "aws_instance" "my-dual-instances" {
  count         = 2  # count will create number instances based on numbers defined
  ami           = "ami-04b4f1a9cf54c11d0"
  instance_type = "t2.micro"
  key_name      = "terraform-kp-1"
  }
}
```

Run terraform validate , plan & apply

From GUI verify resource (2 ec2 instances) getting created in us-west-2 region.

destroy resource

VPC (Virtual private cloud) :

1. **VPC** is a private network in the cloud (like AWS) where you can securely run your resources (e.g., servers, databases) with full control over network settings like IP addresses, subnets, and security.
2. **Security Control**: With a VPC, you can control who can access your resources by setting up firewalls (Security Groups) and restricting access from the internet.
3. **Isolation**: It keeps your cloud resources isolated from other users' resources in the cloud, giving you a private environment for your applications and data.
4. **Customizable Network**: You can create multiple subnets, choose IP ranges, and configure routing, just like in a traditional on-premise data center.
5. **Why Use VPC**: Without a VPC, your cloud resources will be exposed to the public internet, which can make them vulnerable to security threats. VPC ensures that your resources are secure, private, and well-organized.

What happens if you don't use a VPC?

- Your resources may be **publicly accessible** and at risk of attacks or unauthorized access.
- You lose control over **network configuration** and **security**.
- Your resources may be less organized, leading to management difficulties.

Summary: VPC gives you the **privacy**, **security**, and **control** you need for running your cloud resources safely.

Ex3. Write an terraform file to Create a VPC in aws cloud also use tag as dev-vpc & cidr range 10.0.0.0/16

```
provider "aws" {
  region = "us-east-1"
}
resource "aws_vpc" "dev-vpc" {
   cidr_block = "10.0.0.0/16" # The VPC will have IP addresses from 10.0.0.0 to 10.0.255.255
   tags = {
     Name = "dev-vpc"
   }
}
```

Terraform plan & apply

From GUI verify resource (vpc) getting created in us-east-1 region with Name(tag) as dev-vpc

Then destroy resource

Note:

- In AWS, the key **Name** is commonly used to represent the name of the resource.
- If we use **Name** tag then will be visible in the AWS Management Console
- We can use any tag name as we want

Ex4. Write an terraform file to Create a VPC in aws cloud & cidr range 10.0.0.0/16 also create subnet dev-subnet with cidr range 10.0.10.0/24

```
provider "aws" {
  region = "us-east-1"
}

# Below block creates a VPC in AWS with the name "qa-vpc"
resource "aws_vpc" "qa-vpc" {
   cidr_block = "10.0.0.0/16"  # VPC will have IP addresses ranging from 10.0.0.0 to 10.0.255.255
   tags = {
     Name = "qa-vpc"  # VPC is given a tag with key "Name" and value "qa-vpc"
   }
}

# Below block creates a new subnet within the VPC created above (qa-vpc)
resource "aws_subnet" "qa-subnet" {
   vpc_id = aws_vpc.qa-vpc.id  # References the ID of the "qa-vpc" VPC
   cidr_block = "10.0.10.0/24"  # The subnet will use IP addresses from 10.0.10.0 to 10.0.10.255 (256 addresses)
}
```

How to selectively delete a resource without destroying all resources in main.tf ?

Terraform destroy command by default will delete all resources defined in .tf files, in case if we want to select & destroy specific resource

Syntax:

```
terraform destroy -target=<provider>_<resourcetype>.<resource-name>
```

To selectively delete only subnet qa-subnet created above

```
terraform destroy -target=aws_subnet.qa-subnet
```

Note: alternatively, you can edit configuration file & delete those lines & run terraform apply but its not recommended as we lose tracking of the changes done to resources.

Terraform State: (actual / current state & desired state)

Terraform keeps track of your current state of infrastructure & fetches the desired state (from our config file) main.tf & tries to match current state with desired state when we run terraform apply / destroy commands.

Name of the state file → terraform.tfstate ← actual/ current state
Desired state → in configuration file ← main.tf

What is a Terraform State File?

- The state file is a **JSON file** (terraform.tfstate) that stores information about the infrastructure Terraform manages.
- Terraform uses this state file to determine what resources exist and how they should be modified when you run terraform apply.

How terraform handles state file?

Once we run terraform plan / destroy command, In background terraform will calculate what changes it has to do to bring your current state to the desired state.

What does state file contains?

State file stores information about each resource's:

- ID
- Attributes
- Dependencies
- Metadata

State File Modifications:

- **Don't Manually Edit**: The state file should never be manually edited, as it can lead to inconsistencies and errors in your infrastructure. Always use Terraform commands to modify the state.
- **State Commands**: Terraform provides commands to modify and manage the state safely, including:
    - **terraform state list**: List resources in the state file.
    - **terraform state show**: Show detailed information about a resource in the state file.
    - **terraform state rm**: Remove a resource from the state file (use with caution!).
    - **terraform state mv**: Move resources between modules in the state.

What is `terraform.tfstate.backup` file?

- It is the Backup of State File
- The `terraform.tfstate.backup` file contains the previous version of the `terraform.tfstate` file.
- This backup file helps you recover from accidental corruption, data loss, or errors during Terraform operations.